

Bloque 2: JPA básico

Objetivo del bloque

Pasar del concepto a la práctica real: definir una entidad, configurar JPA y entender cómo se realizan las operaciones básicas sobre la base de datos usando objetos.

1. Qué papel juega JPA en la práctica

En el bloque anterior se vio que ORM es una técnica.
Ahora necesitamos una forma concreta de aplicarla en Java.

Aquí entra JPA.

JPA define:

- Cómo se mapean clases a tablas
- Cómo se gestionan los objetos persistentes
- Cómo se realizan operaciones sobre la base de datos

Pero JPA no ejecuta nada por sí sola.
Necesita una implementación.

2. Hibernate como implementación

Hibernate es el motor que ejecuta las operaciones definidas por JPA.

Flujo real:

1. El desarrollador escribe código usando JPA
2. Hibernate interpreta ese código
3. Hibernate genera y ejecuta SQL en la base de datos

Idea clave:

El desarrollador no trabaja directamente con SQL, pero el SQL sigue existiendo.

3. Configuración en Java puro: persistence.xml

En aplicaciones sin frameworks, toda la configuración se define en un archivo llamado:

`persistence.xml`

Este archivo describe:

- Cómo conectarse a la base de datos
- Qué proveedor (Hibernate) se va a usar
- Cómo se comporta el ORM

Ejemplo:

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    version="3.0">

    <persistence-unit name="miUnidad">

        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/tienda"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password" value="1234"/>
            <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>

            <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/>
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
        </properties>

    </persistence-unit>
</persistence>
```

Conceptos importantes aquí:

- **persistence-unit**: define una unidad de persistencia (configuración completa)
 - **dialect**: adapta Hibernate al tipo de base de datos
 - **hbm2ddl.auto**:
 - **update**: sincroniza tablas automáticamente
 - **show_sql**: muestra el SQL generado
-

4. Entidades en JPA

Una entidad es una clase Java que JPA va a mapear a una tabla.

Ejemplo:

```
import jakarta.persistence.*;

@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nombre;
    private String email;

    public Usuario() {}

    // getters y setters
}
```

5. Anotaciones fundamentales

@Entity

Marca la clase como entidad.

Sin esta anotación, JPA ignora la clase.

@Id

Define la clave primaria.

Es obligatoria en cualquier entidad.

@GeneratedValue

Indica que el identificador se genera automáticamente.

Tipos comunes:

- **IDENTITY:** lo genera la base de datos (auto-increment)
-

@Column (opcional)

Permite personalizar columnas.

Si no se usa, JPA asume que el nombre del atributo coincide con la columna.

6. EntityManager: el núcleo del sistema

El EntityManager es el componente que gestiona la persistencia.

Es el intermediario entre:

- la aplicación Java
- la base de datos

Se encarga de:

- guardar objetos
 - recuperarlos
 - eliminarlos
 - sincronizar cambios
-

7. Creación del EntityManager

```
EntityManagerFactory emf =  
    Persistence.createEntityManagerFactory("miUnidad");
```

```
EntityManager em = emf.createEntityManager();
```

Conceptos:

- **EntityManagerFactory:** objeto pesado, se crea una vez
 - **EntityManager:** objeto ligero, se usa para operaciones
-

8. Operaciones básicas de persistencia

8.1 Persistir un objeto

```
em.getTransaction().begin();
```

```
Usuario u = new Usuario();  
u.setNombre("Ana");  
u.setEmail("ana@email.com");
```

```
em.persist(u);
```

```
em.getTransaction().commit();
```

Qué ocurre internamente:

- El objeto pasa a estar gestionado por JPA
 - Hibernate genera un INSERT
 - Se guarda en la base de datos
-

8.2 Buscar un objeto

```
Usuario u = em.find(Usuario.class, 1);
```

Qué ocurre:

- Hibernate genera un SELECT
 - Convierte la fila en un objeto Usuario
-

8.3 Actualizar un objeto

```
em.getTransaction().begin();
```

```
Usuario u = em.find(Usuario.class, 1);  
u.setNombre("Nuevo nombre");
```

```
em.getTransaction().commit();
```

Idea clave:

No existe método update.

JPA detecta automáticamente los cambios en objetos gestionados y los sincroniza con la base de datos.

8.4 Eliminar un objeto

```
em.getTransaction().begin();
```

```
Usuario u = em.find(Usuario.class, 1);  
em.remove(u);
```

```
em.getTransaction().commit();
```

Internamente:

- Hibernate genera un DELETE
-

9. Transacciones

Toda operación que modifica datos requiere una transacción.

Estructura:

```
em.getTransaction().begin();
```

```
// operación
```

```
em.getTransaction().commit();
```

Sin transacción:

- No se guardan cambios
 - Puede haber errores
-

10. Ciclo básico de uso

Flujo típico en una aplicación:

1. Crear EntityManagerFactory
 2. Crear EntityManager
 3. Iniciar transacción
 4. Realizar operaciones
 5. Confirmar transacción
 6. Cerrar recursos
-

11. Qué está ocurriendo realmente (visión interna)

Aunque el código parece simple:

```
em.persist(usuario);
```

Internamente ocurre:

1. Hibernate detecta la entidad
2. Genera SQL (INSERT)
3. Ejecuta la consulta
4. Sincroniza el estado del objeto

Esto es importante porque:

- ORM no elimina SQL
 - lo oculta y lo automatiza
-

12. Idea final del bloque

En este bloque se da el salto de teoría a práctica:

- Se define una entidad
- Se configura la persistencia
- Se utiliza EntityManager
- Se realizan operaciones básicas sin escribir SQL

13. Configuración para el proyecto

Click en: **New Project**

Selecciona:

- **Maven**

Añade las dependencias que vamos a usar:

```
<dependencies>
<!-- JPA API -->
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.1.0</version>
</dependency>
<!-- Hibernate (implementación JPA) -->
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.0.Final</version>
</dependency>
```

```

<!-- MySQL Driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.3.0</version>
</dependency>
</dependencies>

```

Ahora creamos persistence.xml

Ruta EXACTA:

src/main/resources/META-INF/persistence.xml

Contenido:

```

<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  version="3.0">
  <persistence-unit name="miUnidad">
    <properties>
      <property name="jakarta.persistence.jdbc.url"
        value="jdbc:mysql://localhost:3306/NOMBRE DE LA BASE DE DATOS"/>

      <property name="jakarta.persistence.jdbc.user" value="root"/>
      <property name="jakarta.persistence.jdbc.password" value="root"/>
      <property name="jakarta.persistence.jdbc.driver"
        value="com.mysql.cj.jdbc.Driver"/>

      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>

```

Crea clase:

```

import jakarta.persistence.*;
@Entity
public class Usuario {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int id;
  private String nombre;
  public Usuario() {}
  public Usuario(String nombre) {
    this.nombre = nombre;
  }

  public int getId() { return id; }
  public String getNombre() { return nombre; }
  public void setNombre(String nombre) {
    this.nombre = nombre;
  }
}

```



```
}
```

Crear Main Y probar

```
import jakarta.persistence.*;
public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("miUnidad");
        EntityManager em = emf.createEntityManager();
        // INSERT
        em.getTransaction().begin();
        Usuario u = new Usuario("Gabriel");
        em.persist(u);
        em.getTransaction().commit();
        System.out.println("Usuario guardado con ID: " + u.getId());
        // SELECT
        Usuario encontrado = em.find(Usuario.class, u.getId());
        System.out.println("Nombre: " + encontrado.getNombre());
        em.close();
        emf.close();
    }
}
```

14. Ejercicio de ejemplo (CRUD completo real)

Clase Usuario

```
import jakarta.persistence.*;

@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nombre;
    private String email;

    public Usuario() {}

    public Usuario(String nombre, String email) {
```

```
        this.nombre = nombre;
        this.email = email;
    }

    // getters y setters
}
```

Clase Main

```
import jakarta.persistence.*;

public class Main {

    public static void main(String[] args) {

        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("miUnidad");

        EntityManager em = emf.createEntityManager();

        // CREATE
        em.getTransaction().begin();

        Usuario u = new Usuario("Ana", "ana@mail.com");
        em.persist(u);

        em.getTransaction().commit();

        // READ
        Usuario encontrado = em.find(Usuario.class, u.getId());
        System.out.println(encontrado.getNombre());

        // UPDATE
        em.getTransaction().begin();

        encontrado.setNombre("Ana Actualizada");

        em.getTransaction().commit();

        // DELETE
        em.getTransaction().begin();

        em.remove(encontrado);

        em.getTransaction().commit();

        em.close();
        emf.close();
    }
}
```

}
}

EJERCICIOS

1. CRUD completo de Usuario

Implementar:

- persist
- find
- update
- remove

2. Insertar múltiples registros en una transacción

Crear 5 productos en una sola transacción.

3. Buscar entidad por ID

Implementar búsqueda y mostrar datos en consola.

4. Actualización de campo

Cambiar el email de un usuario existente.

5. Eliminación segura

Eliminar un usuario solo si existe (validación con find).

6. Crear entidad Libro y persistir varios registros

Libro: id, titulo, autor, precio.

7. Simular error de transacción

Hacer persist de varios objetos y forzar excepción antes del commit.

8. Crear método reutilizable de persistencia

Método `guardar(Object entity)` usando EntityManager.

9. Consultar todos los registros

JPQL:

```
SELECT u FROM Usuario u
```

10. Mini sistema de consola

Menú:

- Crear usuario
- Listar usuarios
- Eliminar usuario